

INVENTORY/ACCOUNT MANAGEMENT SYSTEM

Complete Setup and Run Guide

Beginner-friendly instructions for running the project from GitHub

Project type	Full-stack Inventory Management System
Frontend	React + Vite + Axios + React Router
Backend	Node.js + Express
Database	MongoDB / MongoDB Atlas
Authentication	JWT + bcryptjs

This guide focuses on the files and settings you need to install, configure, and run the system.

NOTE: Create your own backend/.env file and enter your own MongoDB connection details and JWT secret. Never share private credentials through GitHub.

1. What This Guide Is For

This document explains how a new user or developer can take the Inventory Management System from a GitHub repository, install the required software, connect MongoDB, create the environment file, start the backend and frontend, and open the system in a browser.

The instructions are intentionally written in simple student-friendly language. You do not need to understand the whole codebase before running it.

1.1. What you will do

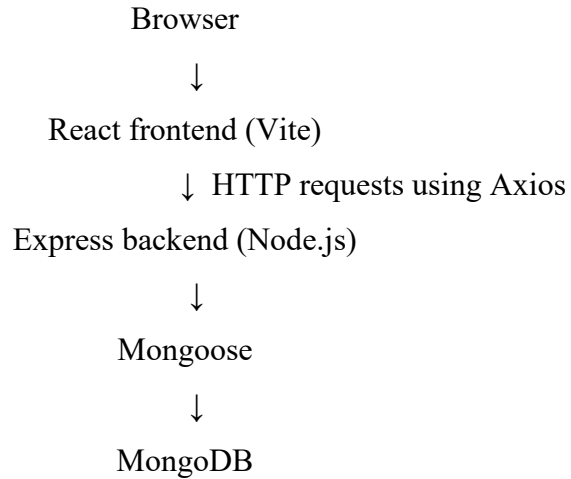
- Install Node.js and make sure npm is available.
- Clone the GitHub repository.
- Open the project in VS Code or another editor.
- Create the backend/.env file.
- Create/connect a MongoDB database.
- Install backend packages and start the Express server.
- Install frontend packages and start the React/Vite application.
- Open the frontend in the browser and create a company/admin account.

2. Quick Understanding of the System

The project is split into two main applications: a React frontend and a Node/Express backend. MongoDB stores the actual application data.

Part	Folder	Main job	How it runs
Frontend	frontend/	User interface and pages	npm run dev
Backend	backend/	API, authentication and business logic	npm run dev
Database	MongoDB	Stores companies, admins, products, sales, purchases, etc.	MongoDB Atlas or local MongoDB
Environment	backend/.env	Stores MongoDB URL, JWT secret and optional seed credentials	Loaded automatically by dotenv

2.1. Simple data flow



The frontend sends requests to `http://localhost:5000/api`. The backend checks the JWT token for protected requests and then reads/writes data in MongoDB.

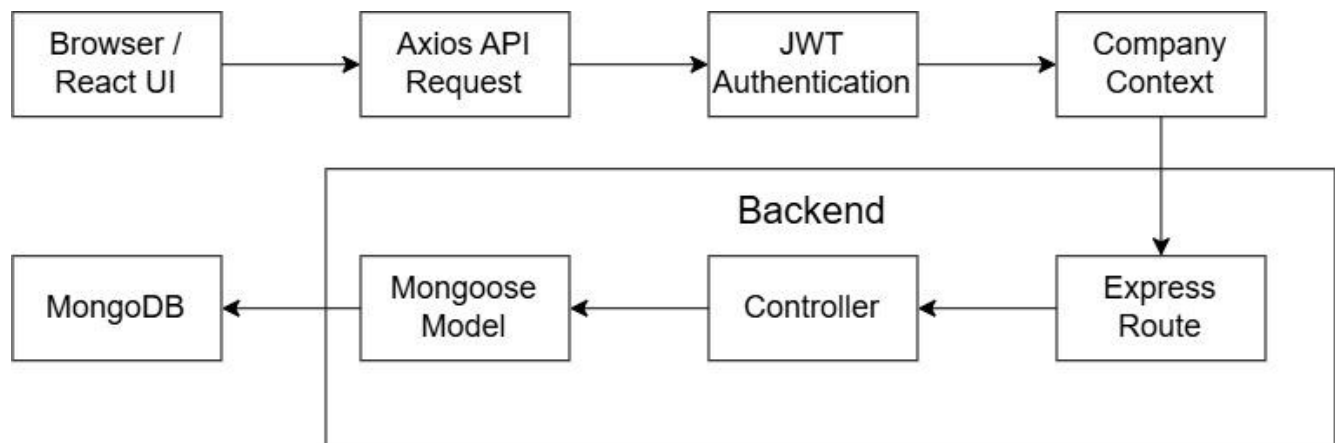


Figure 1: Overall Architecture of System

3. Project Structure

After cloning the repository, the important structure is approximately:

```
INVENTORY-MANAGEMENT-SYSTEM-multitenant/
├── backend/
│   ├── .env                ← you create/edit this locally
│   ├── .env.example        ← environment variable template
│   ├── package.json
│   ├── package-lock.json
│   └── src/
│       ├── config/
│       │   └── db.js
│       ├── controllers/
│       ├── middleware/
│       ├── models/
│       ├── plugins/
│       ├── routes/
│       ├── services/
│       ├── utils/
│       └── server.js
└── frontend/
    ├── index.html
    ├── package.json
    ├── package-lock.json
    └── src/
        ├── components/
        ├── context/
        ├── pages/
        ├── services/
        ├── styles/
        └── utils/
```

3.1. What the main backend folders do

config: Database connection code.

controllers: Contains the actual actions for API requests, such as creating products or invoices.

middleware: Checks things that should happen before a request reaches a controller. The main example is JWT authentication.

models: Mongoose schemas describing the MongoDB documents used by the system.

plugins: Shared Mongoose logic. The companyScope plugin keeps normal business data separated by company.

routes: Defines API URLs and connects them to controllers.

services: Reusable business logic, especially authentication.

utils: Helper code, tenant context, calculations, seed script and other utilities.

server.js: Starts Express, enables CORS/JSON parsing, registers routes, connects MongoDB and listens on port 5000.

3.2. What the main frontend folders do

components: Reusable interface pieces such as the layout, protected route and date input.

context: Authentication state shared across the React application.

pages: Main screens such as Login, Dashboard, Products, Customers, Sales, Payments and Balance Sheet.

services: Axios calls to the backend API.

styles: CSS files for the application.

utils: Date conversion and other frontend helper functions.

4. Software You Need Before Starting

Install these before cloning/running the project:

Software	Recommended requirement	Why
Node.js	Node 20.19+ or Node 22.12+	Required by the Vite version used by the frontend.
npm	Comes with Node.js	Installs project dependencies and runs scripts.
Git	Current version	Used to clone the GitHub repository.
VS Code	Recommended	Makes editing .env and code easier.
MongoDB	MongoDB Atlas recommended	Stores application data.
Web browser	Chrome/Edge/Firefox	Used to access the frontend.

Use Node.js 20.19+ or 22.12+. A recent LTS version is recommended.

4.1. Check Node.js and npm

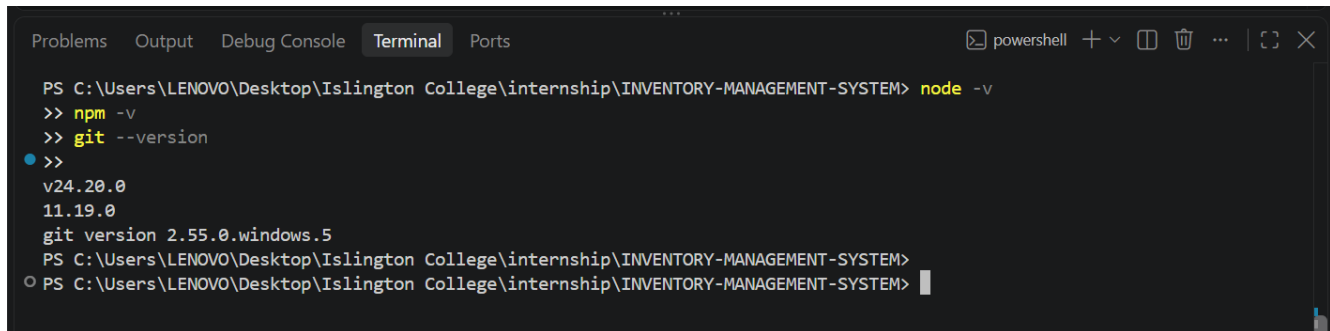
node

-v

npm

-v

git --version

A screenshot of a PowerShell terminal window. The title bar shows 'powershell' and standard window controls. The terminal content shows the following commands and output:

```
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM> node -v
>> npm -v
>> git --version
v24.20.0
11.19.0
git version 2.55.0.windows.5
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM>
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM>
```

Figure 2: Checking Node.js and npm versions

5. Clone the Project from GitHub

First decide where you want to keep the project. For example, on Windows you can use your D: drive.

cd D:

git clone <YOUR-GITHUB-REPOSITORY-URL>

cd INVENTORY-MANAGEMENT-SYSTEM-multitenant

Repo: <https://github.com/manisha-shah123/INVENTORY-ACCOUNTANCY-MANAGEMENT-SYSTEM.git>

6. Set Up MongoDB

The backend connects to MongoDB using the MONGO_URI value in backend/.env. The supplied .env.example shows a MongoDB Atlas connection string, so Atlas is the easiest setup for a new user.

6.1 Create a MongoDB Atlas cluster

1. Open MongoDB Atlas and sign in/create an account.
2. Create a cluster if you do not already have one.

cloud.mongodb.com/v2/6ac4be47421d4f8aac7e4211#/overview

Project 0 Overview

Create a cluster

Choose your cloud provider, region, and specs.

+ Create

Support Plan: Basic Plan

CHAT WITH SUPPORT VIEW SUPPORT

Featured Resources

- GENERAL
- Get Started with Atlas
- Reference MongoDB Documentation
- Learn MongoDB Skills
- Atlas Learning Hub

New On Atlas

Learn about the latest feature enhancements c

cloud.mongodb.com/v2/6ac4be47421d4f8aac7e4211#/clusters/starterTemplates?source=Atlas+Home+Page

PROJECT Project 0

M10 \$0.10/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU	MAX CONN.
10 GB	2 GB	2 vCPUs	1500

Compare features

Flex From \$0.011/hour Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU	MAX CONN.
5 GB	Shared	Shared	500

Compare features

Free

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU	MAX CONN.
512 MB	Shared	Shared	500

Compare features

Configurations

Name

You cannot change the name once the cluster is created.

Cluster0

Provider

aws Google Cloud Azure

Region

Mumbai (asia-south1) ★

★ Recommended Low carbon emissions

Quick setup

- Automate security setup
- Preload sample dataset

Pricing

\$0/hour

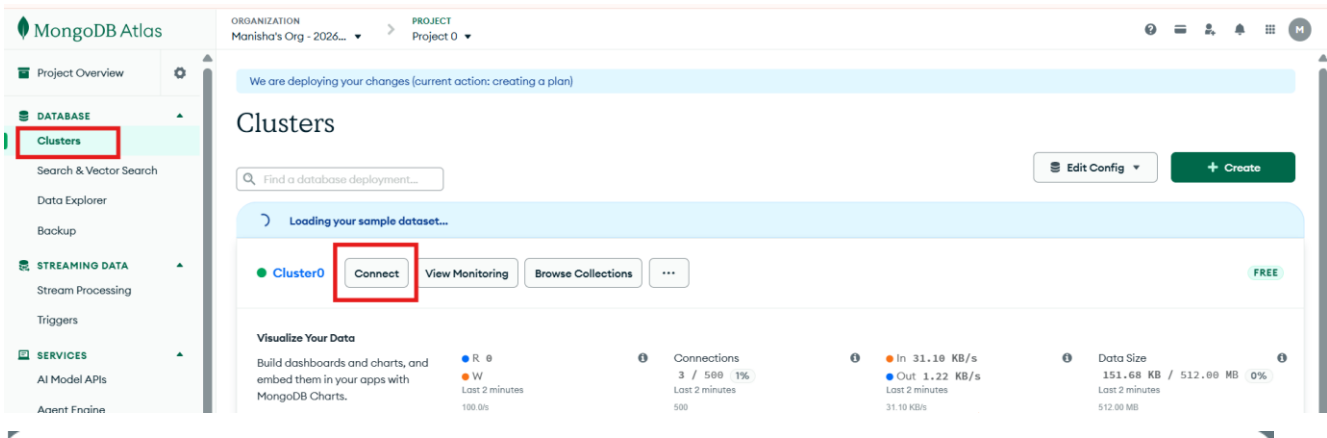
Free forever! Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

I'll do this later

Go to Advanced Configuration

Create Deployment

3. Create a database user with a username and password.
4. Add your current IP address in Network Access. For a temporary development setup, Atlas may also offer an option to allow access from anywhere; this is less restrictive and should be used carefully.
5. Open the cluster's Connect option and choose the Drivers connection method.



Connect to Cluster0



Connect to your application



Drivers and Client Libraries

Connect using MongoDB drivers (e.g. Node.js, Go) or popular libraries (e.g. Mongoose, Prisma)



Access your data through tools



Atlas SQL NEW

Easily connect SQL tools to Atlas for data analysis and visualization. Atlas SQL now supports direct connections to dedicated clusters!



Compass

Explore, modify, and visualize your data with MongoDB's GUI



Shell

Quickly add & update data using MongoDB's Javascript command-line interface



MongoDB for VS Code

Work with your data in MongoDB directly from your VS Code environment



Go Back

Close

6. Copy the MongoDB connection string.

Connect to Cluster0



Connecting with MongoDB Client Library

Language

JavaScript

Client Library

Node.js Driver

The official library for connecting to MongoDB and performing CRUD, aggregation, and administrative operations from any Node.js application. [Node.js Driver installation instructions](#)

Configure connection string

Installation instructions

COMPRESSION

☐ Enable network compression

Reduce MongoDB network traffic by compressing messages between your application and cluster. Some compressors may require additional driver dependencies. Click [here](#) for more information.

USE THIS CONNECTION STRING IN YOUR APPLICATION

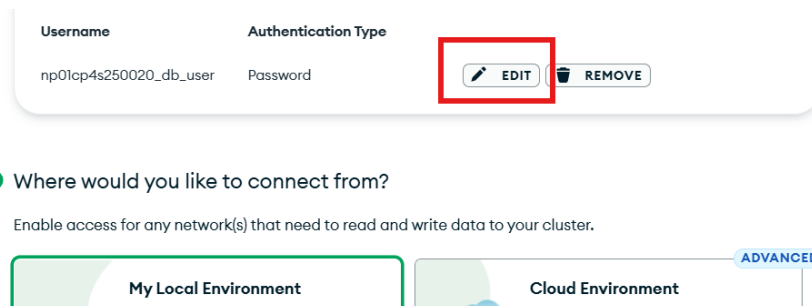
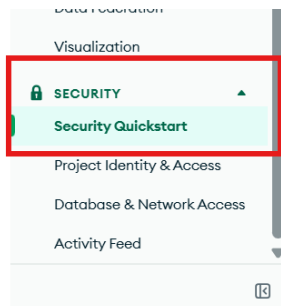
☐ Legacy URI String ?

```
mongodb+srv://np01cp4s250020_db_user:
<db_password>@cluster0.lwx7y1t.mongodb.net/?appName=Cluster0
```

Replace **<db_password>** with the password for the **np01cp4s250020_db_user** database user. Ensure any option params are [URL encoded](#).

7. For the **<db_password>**

- Go to **Security Quickstart** on the side navbar
- Click on edit and add a password
- Use the password in the mongodb URI string in backend/.env file



The database name can be included at the end of the connection string. The project's .env.example uses inventory_db.

mongodb+srv://<username>:<password>@<cluster-host>/inventory_db?appName=Cluster0

6.2 What the application creates in MongoDB

The application uses Mongoose models for companies, admins, products, clients, invoices, payments, receipts, purchases, expenses, stock movements, balance information and related attributes.

You do not need to manually create every collection before starting. When the application creates documents through Mongoose, MongoDB will create the relevant collections as needed.

Important: A new user should create the MongoDB database/user and connection string, then let the application create its collections. The first company/admin can be created through the Register page.

7. Create the Backend .env File

The backend uses dotenv, so it reads environment variables from a file named .env located directly inside the backend folder.

You should have:

INVENTORY-MANAGEMENT-SYSTEM-multitenant/

```
└─ backend/  
    └─ .env  
    └─ .env.example  
    └─ src/
```

7.1 Easy Windows method

cd D:\path\to\INVENTORY-MANAGEMENT-SYSTEM\backend

copy .env.example .env

Then open backend/.env in VS Code and edit it.

7.2 Minimum .env for normal registration

PORT=5000

MONGO_URI=mongodb+srv://<username>:<password>@<cluster-host>/?appName=Cluster0

JWT_SECRET=replace_this_with_a_long_random_secret

These are the values the normal application flow needs. The user can then open the Register page and create a company/admin account.

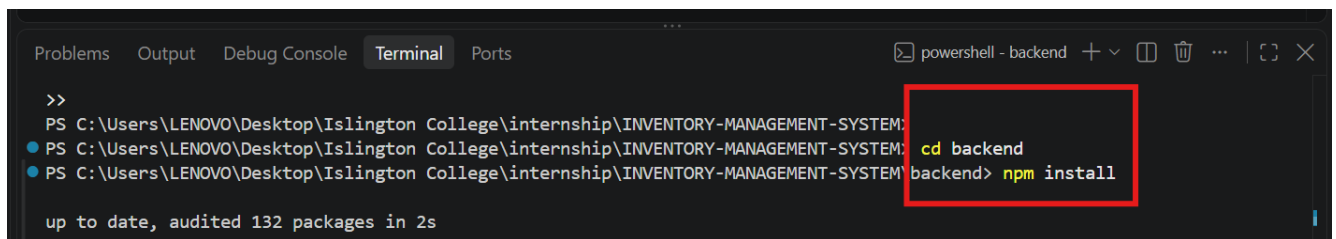
8. Install and Run the Backend

1. Open a terminal and go into the backend folder.

cd D:\path\to\INVENTORY-MANAGEMENT-SYSTEM-multitenant\backend

2. Install the backend packages:

npm install

A screenshot of a PowerShell terminal window. The title bar reads 'powershell - backend'. The terminal shows the following commands and output:

```
>>  
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM>  
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM> cd backend  
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM\backend> npm install  
up to date, audited 132 packages in 2s
```

The commands 'cd backend' and 'npm install' are highlighted with a red rectangular box.

3. Start the backend in development mode:

npm run dev

The backend package.json defines npm run dev as nodemon src/server.js.

When it works, you should see messages similar to:

Server running on port 5000

Connected to MongoDB

```
Run `npm audit` for details.
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM\backend> npm run dev

> inventory-management-backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
◇ injected env (3) from .env // tip: secrets for agents [www.dotenvx.com]
Server running on port 5000
Connected to MongoDB
```

Important: The exact order of the two messages may vary because the Express server starts listening while the database connection is established asynchronously.

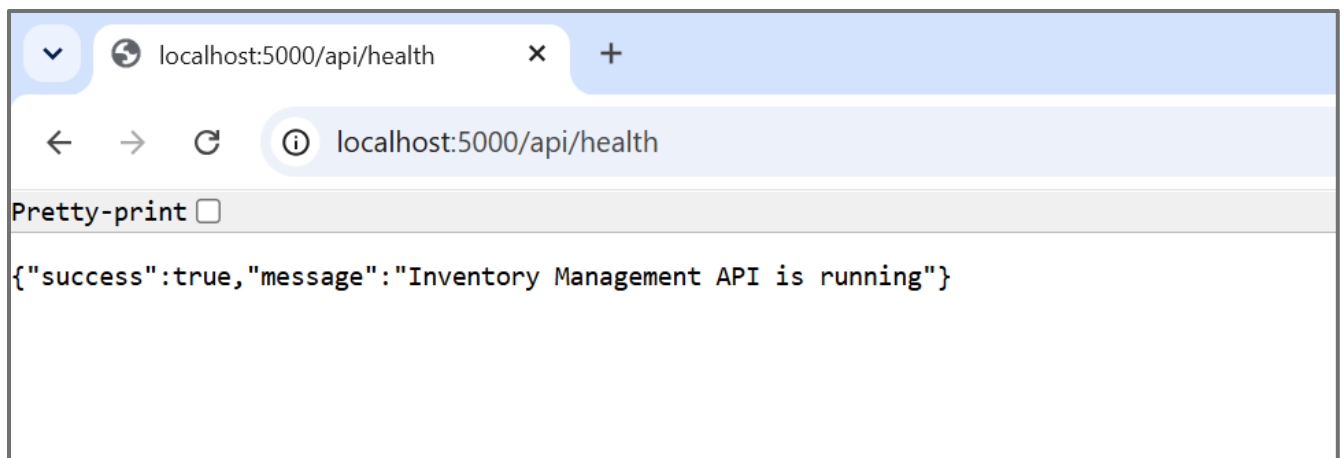
8.1 Test the backend directly

Open this URL in your browser:

`http://localhost:5000/api/health`

The backend has a health endpoint that should return JSON similar to:

```
{
  "success": true,
  "message": "Inventory Management API is running"
}
```



9. Install and Run the Frontend

Keep the backend terminal running. Open a second terminal window and go to the frontend folder.

```
cd D:\path\to\INVENTORY-MANAGEMENT-SYSTEM-multitenant\frontend
```

Install frontend dependencies:

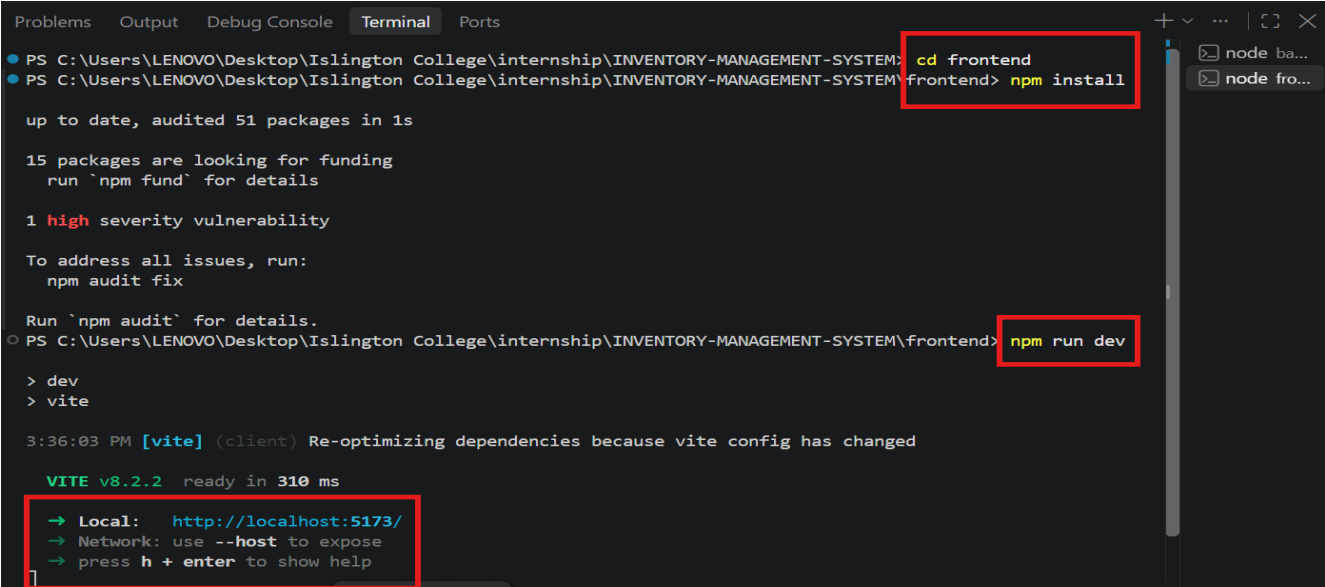
```
npm install
```

Start the React/Vite development server:

```
npm run dev
```

Vite will print a Local URL, normally similar to:

```
http://localhost:5173/
```



```
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM> cd frontend
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM\frontend> npm install

up to date, audited 51 packages in 1s

15 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\LENOVO\Desktop\Islington College\internship\INVENTORY-MANAGEMENT-SYSTEM\frontend> npm run dev

> dev
> vite

3:36:03 PM [vite] (client) Re-optimizing dependencies because vite config has changed

VITE v8.2.2 ready in 310 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

NOTE: Press **ctrl** and click on the <http://localhost:5173> together

9.1. Important: frontend API address

The frontend uses the backend address in frontend/src/services/api.js:

```
const API_URL = "http://localhost:5000/api";
```

Because of this, the backend must normally run on port 5000 when you use the project as supplied. There is no frontend .env file used for the API URL in the supplied code.

10. Open and Use the System

Once both terminals are running:

Backend → **http://localhost:5000**

Frontend → **http://localhost:5173**

Open the frontend URL in your browser.

The main authentication flow is:

Frontend

↓

/login

↓

Login API (/api/auth/login)

↓

JWT token returned

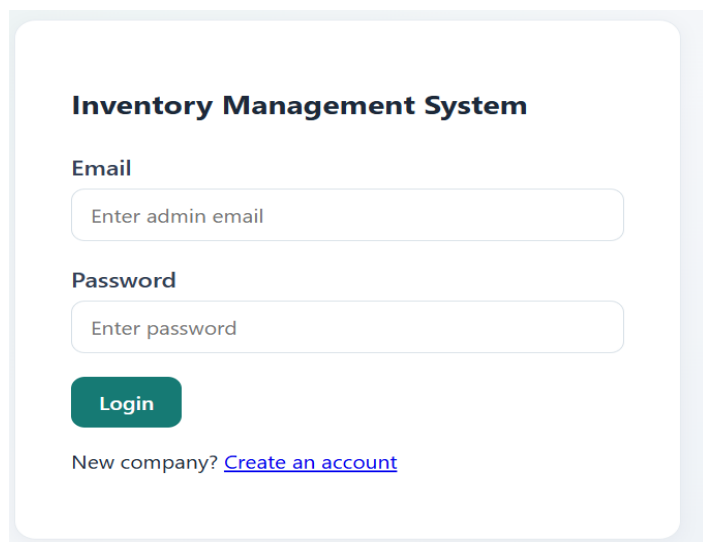
↓

Token stored in browser localStorage

↓

Protected pages become available

The application also provides a Register page. A new company can be created from /register. Registration creates the company and its first admin, hashes the password using bcryptjs, creates a JWT, and logs the user into the dashboard.



The screenshot shows a login form for the 'Inventory Management System'. It features a title, an 'Email' field with a placeholder 'Enter admin email', a 'Password' field with a placeholder 'Enter password', and a green 'Login' button. At the bottom, there is a link for 'New company? Create an account'.

Inventory Management System

Email

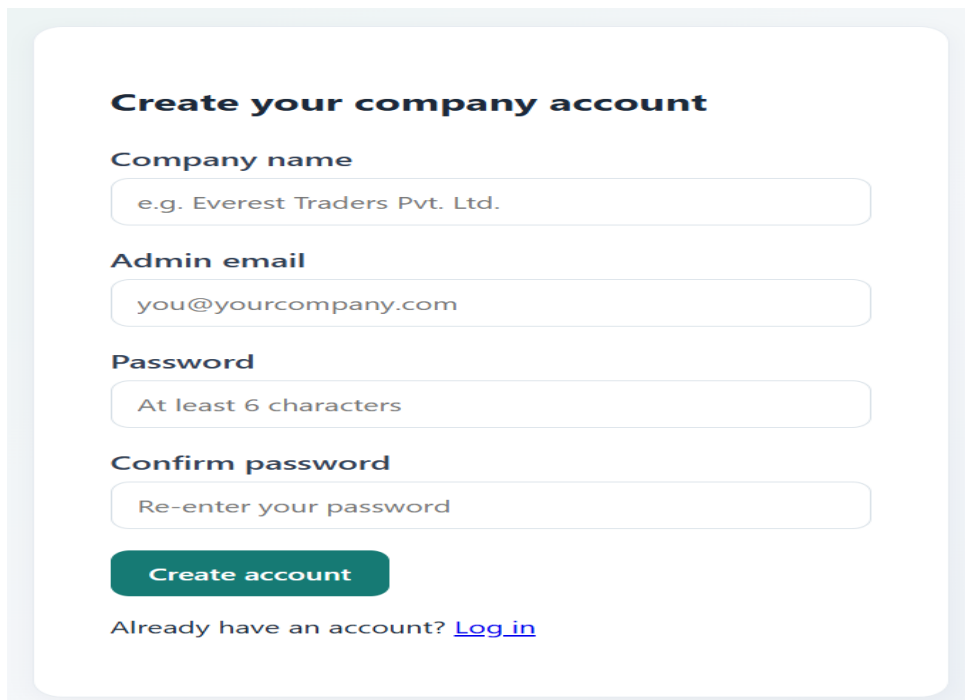
Enter admin email

Password

Enter password

Login

New company? [Create an account](#)



The image shows a web form titled "Create your company account". It contains four input fields: "Company name" with a placeholder "e.g. Everest Traders Pvt. Ltd.", "Admin email" with a placeholder "you@yourcompany.com", "Password" with a placeholder "At least 6 characters", and "Confirm password" with a placeholder "Re-enter your password". Below the fields is a green "Create account" button. At the bottom, there is a link "Already have an account? [Log in](#)".

Create your company account

Company name
e.g. Everest Traders Pvt. Ltd.

Admin email
you@yourcompany.com

Password
At least 6 characters

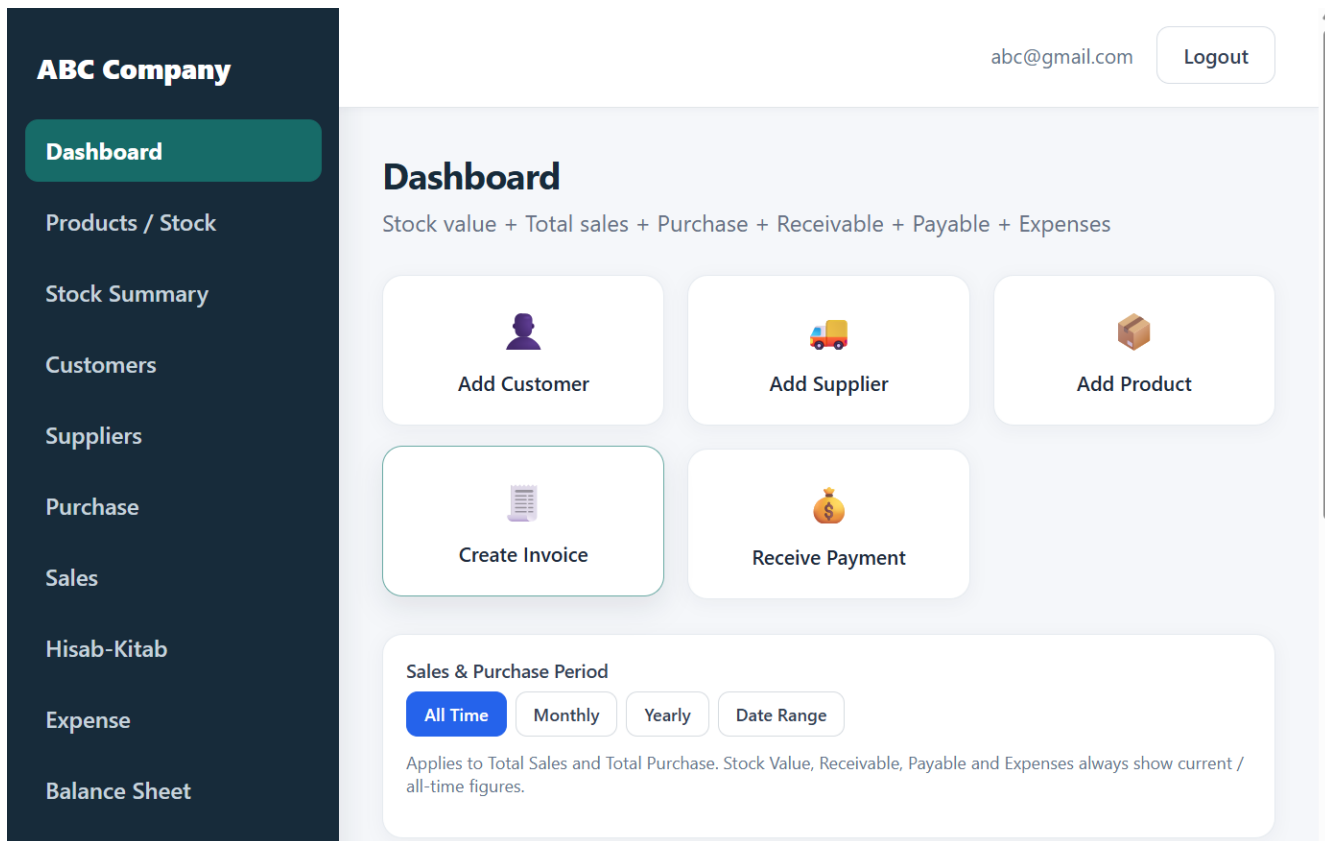
Confirm password
Re-enter your password

Create account

Already have an account? [Log in](#)

10.1 First-time setup using Register

8. Open the frontend URL.
9. Go to Create an account / Register.
10. Enter a company name.
11. Enter an admin email.
12. Enter a password of at least 6 characters.
13. Confirm the password.
14. Click Create account.
15. The application sends the registration request to the backend.
16. The backend creates the company and admin in MongoDB.
17. The password is hashed with bcryptjs; the plain password is not stored as the database password.
18. A JWT is returned and the frontend stores it in localStorage.
19. You are redirected to the dashboard.



11. Main Features Available After Login

The supplied frontend has routes/pages for the following major areas:

Area	Example URL	Purpose
Dashboard	/dashboard	Summary such as stock value, sales, purchases, receivable, payable and expenses.
Products / Stock	/dashboard/products	Manage products and stock adjustments.
Stock Summary	/dashboard/stock-summary	View stock summary.
Customers	/dashboard/customers	Manage customers and customer reports/statements.
Suppliers	/dashboard/suppliers	Manage suppliers.
Purchase	/dashboard/purchases	Create and manage purchases and purchase returns.
Sales	/dashboard/sales	Create, view and edit invoices/sales.

Hisab-Kitab	/dashboard/hisab-kitab	Payments, received payments and receipts.
Expense	/dashboard/expenses	Create and manage expenses.
Balance Sheet	/dashboard/balance-sheet	Opening balances and balance-sheet entries.

The dashboard also supports period filtering for sales and purchases, including monthly/yearly selections and date ranges, with BS/AD date handling in the frontend.

End of Guide